Blockchain : Experiment - 10

Aim:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode.

Theory:

Hyperledger Fabric Network

Hyperledger Fabric is a permissioned blockchain platform that enables multiple organizations to maintain a shared, tamper-resistant ledger. It supports smart contracts, known as chaincode, which define business rules and execute transactions. In this experiment, Hyperledger Fabric is used to establish a blockchain network, create a channel, and perform asset-management operations. The network consists of peer nodes, an ordering service, and participating organizations that collaborate to validate and record transactions.

Hyperledger Fabric Network Setup

The Fabric network is configured using Docker containers, Fabric binaries, and the official fabric-samples repository. Docker runs the blockchain components, while the Fabric command-line interface (CLI) is used to configure the network and interact with the ledger. A channel named mychannel is created to provide a shared ledger for transactions. The participating peers maintain ledger copies and execute the chaincode according to the network's endorsement policies.

Chaincode and Smart Contracts

Chaincode is the smart contract mechanism in Hyperledger Fabric that implements business logic and defines how assets are managed. In this experiment, the asset-transfer-basic JavaScript chaincode is deployed on the Fabric network. It provides functions to initialize the ledger, create assets, retrieve asset details, and transfer ownership. The chaincode ensures that asset operations follow the defined rules and that valid transactions update the ledger.

Asset Management Using Chaincode

The asset-management process demonstrates how digital assets can be created, queried, and transferred on a blockchain. Each asset contains attributes such as its ID, color, size, owner, and appraised value.

- InitLedger(): Initializes the ledger with predefined sample assets.
- CreateAsset(): Creates a new asset with specified attributes and assigns an initial owner.
- GetAllAssets(): Retrieves all assets stored in the ledger.
- ReadAsset(): Retrieves the details of a particular asset using its ID.
- TransferAsset(): Changes the ownership of an existing asset.

These functions demonstrate how blockchain transactions maintain asset records and provide a consistent view of the current ownership information.

Transaction Validation and Endorsement

Hyperledger Fabric uses an endorsement process to ensure that transactions satisfy the network's policies before their changes are applied to the ledger. Peer nodes simulate transactions and provide endorsements, while the ordering service organizes transactions into blocks. The peers subsequently validate transactions and commit the blocks to the ledger. Transactions that fail endorsement-policy or validation checks are marked invalid, and their proposed state changes are not applied. This process helps maintain the consistency and integrity of the ledger across participating organizations.

Docker and Fabric CLI Integration
Docker provides an isolated environment for running Fabric components, including peer nodes and the ordering service. The Fabric CLI enables users to start the network, create channels, deploy chaincode, invoke transactions, and query asset records. Commands such as peer chaincode invoke and peer chaincode query are used to execute state-changing operations and retrieve ledger information. The network can be stopped using the provided network script after completing the experiment.

Working of the Asset Management System
The experiment begins by configuring the required dependencies and installing Hyperledger Fabric binaries and Docker images. The Fabric test network is then started, and a channel named mychannel is created. The JavaScript chaincode is deployed and the ledger is initialized with sample assets. Users can retrieve all assets, read individual records, create a new asset with a specified owner, and transfer ownership to another user. Each state-changing operation must satisfy the network's endorsement and validation policies before its changes become part of the ledger. The final asset queries verify the updated ownership, demonstrating the use of Hyperledger Fabric for blockchain-based asset management.

Output:

Install the required dependencies by running:

sudo apt install jq curl git nodejs npm -y

```
# sudo apt install jq curl git nodejs npm -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.16).
curl set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.4).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  libllvm19 libwayland-server0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev eslint fakeroot g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu gyp handlebars javascript-common libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 libatomic1 libauthen-sasl-perl libc-dev-bin libc-devtools libc6-dev libcares2
  libcc1-0 libclone-perl libcrypt-dev libdata-dump-perl libdpkg-perl libencode-locale-perl libfakeroot libfile-basedir-perl libfile-desktopentry-perl
  libfile-fcntllock-perl libfile-listing-perl libfile-mimeinfo-perl libfont-afm-perl libgcc-13-dev libgd3 libgles2 libgomp1 libheif-plugin-aomdec
  libheif-plugin-aomenc libheif1 libhtml-form-perl libhtml-format-perl libhtml-parser-perl libhtml-tagset-perl libhtml-tree-perl libhttp-cookies-perl
  libhttp-daemon-perl libhttp-date-perl libhttp-message-perl libhttp-negotiate-perl libhwasan0 libice6 libio-html-perl libio-socket-ssl-perl
  libio-stringy-perl libipc-system-simple-perl libisl23 libitm1 libjq1 libjs-async libjs-events libjs-inherits libjs-is-typedarray libjs-prettify
  libjs-regenerate libjs-source-map libjs-sprintf-js libjs-typedarray-to-buffer libjs-util liblsan0 liblwp-mediatypes-perl liblwp-protocol-https-perl
  libmailtools-perl libmpc3 libnet-dbus-perl libnet-http-perl libnet-smtp-ssl-perl libnet-ssleay-perl libnode-dev libnode109 libnotify-bin libnotify4
  libonig5 libquadmath0 libre2-10 libsm6 libssl-dev libstdc++-13-dev libtie-ixhash-perl libtimedate-perl libtry-tiny-perl libtsan2 libubsan1 liburi-perl
  libuv1-dev libuv1t64 libwww-perl libwww-robotrules-perl libx11-protocol-perl libxaw7 libxcb-shape0 libxft2 libxkbfile1 libxml-parser-perl
  libxml-twig-perl libxml-xpathengine-perl libxmu6 libxpm4 libxt6t64 libxv1 libxxf86dga1 linux-libc-dev lto-disabled-list make manpages-dev node-abbrev
  node-acorn node-agent-base node-ajv node-ajv-keywords node-ampproject-remapping node-ansi-escapes node-ansi-regex node-ansi-styles node-anymatch
  node-aproba node-archy node-are-we-there-yet node-argparse node-arrify node-assert node-async node-async-each node-auto-bind
  node-babel-helper-define-polyfill-provider node-babel-plugin-add-module-exports node-babel-plugin-lodash node-babel-plugin-polyfill-corejs2
  node-babel-plugin-polyfill-corejs3 node-babel-plugin-polyfill-regenerator node-babel7 node-babel7-runtime node-balanced-match node-base node-base64-js
  node-binary-extensions node-brace-expansion node-braces node-browserslist node-builtins node-busboy node-cacache node-cache-base node-camelcase
  node-caniuse-lite node-chalk node-chokidar node-chownr node-chrome-trace-event node-ci-info node-cjs-module-lexer node-cli-boxes node-cli-cursor
  node-cli-table node-cli-truncate node-cliui node-clone node-clone-deep node-collection-visit node-color-convert node-color-name node-colors
  node-columnify node-commander node-commondir node-concat-stream node-console-control-strings node-convert-source-map node-copy-concurrently node-core-js
  node-core-js-compat node-core-js-pure node-core-util-is node-coveralls node-css-loader node-css-selector-tokenizer node-data-uri-to-buffer
```

Verify each tool individually

- jq --version
- curl --version
- git --version
- node -v
- npm -v

```
# jq --version
jq-1.7
# curl --version
curl 8.5.0 (x86_64-pc-linux-
 libidn2/2.3.7 libpsl/0.21.2
p/2.3 OpenLDAP/2.6.10
Release-Date: 2023-12-06, se
Protocols: dict file ftp ftp
rtmp rtsp scp sftp smb smbs
Features: alt-svc AsynchDNS
libz NTLM PSL SPNEGO SSL thr
# git --version
git version 2.43.0
# node -v
v18.19.1
# npm -v
9.2.0
#
```

Check if docker is connected to WSL by running: docker ps

```
root@SHAKTI:~# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
root@SHAKTI:~#
```

Clone the official repository:

git clone https://github.com/hyperledger/fabric-samples.git

```
root@SHAKTI:~# git clone https://github.com/hyperledger/fabric-samples.git
Cloning into 'fabric-samples'...
remote: Enumerating objects: 16063, done.
remote: Counting objects: 100% (3470/3470), done.
remote: Compressing objects: 100% (641/641), done.
remote: Total 16063 (delta 3045), reused 2829 (delta 2829), pack-reused 12593 (from 3)
Receiving objects: 100% (16063/16063), 24.08 MiB | 5.81 MiB/s, done.
Resolving deltas: 100% (9276/9276), done.
root@SHAKTI:~# cd
.aws/           .cache/         .npm/           fabric-samples/
.azure/         .docker/        .ssh/
root@SHAKTI:~# cd fabric-samples/
root@SHAKTI:~/fabric-samples#
```

Install Fabric binaries and Docker images:

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh

chmod +x install-fabric.sh

./install-fabric.sh docker samples binary

```
root@SHAKTI:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main
/scripts/install-fabric.sh
root@SHAKTI:~/fabric-samples# ls -l install-fabric.sh
-rw-r--r-- 1 root root 11135 Oct 25 10:30 install-fabric.sh
root@SHAKTI:~/fabric-samples# chmod +x install-fabric.sh
root@SHAKTI:~/fabric-samples# ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branch is intended
to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.16 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.16/hyperledger-fab
ric-linux-amd64-2.5.16.tar.gz
===> Will unpack to: /root/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  1  121M    1 1438k    0     0  1883k      0  0:01:06 --:--:--  0:01:06 1883k
```

Verify Docker Images:

docker images | grep fabric

```
root@SHAKTI:~/fabric-samples# docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --
format.
ghcr.io/hyperledger/fabric-baseos:2.5.16    109b4a15b282        257MB         83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16     807493f095f6       1.01GB          257MB
ghcr.io/hyperledger/fabric-orderer:2.5.16   1d918fc7e1e0        182MB         50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16      58979b65744b        232MB         66.8MB
hyperledger/fabric-baseos:2.5               109b4a15b282        257MB         83.2MB
hyperledger/fabric-baseos:2.5.16            109b4a15b282        257MB         83.2MB
hyperledger/fabric-baseos:latest            109b4a15b282        257MB         83.2MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                807493f095f6       1.01GB          257MB
hyperledger/fabric-ccenv:2.5.16             807493f095f6       1.01GB          257MB
hyperledger/fabric-ccenv:latest             807493f095f6       1.01GB          257MB
hyperledger/fabric-orderer:2.5              1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-orderer:2.5.16           1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-orderer:latest           1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-peer:2.5                 58979b65744b        232MB         66.8MB
hyperledger/fabric-peer:2.5.16              58979b65744b        232MB         66.8MB
hyperledger/fabric-peer:latest              58979b65744b        232MB         66.8MB
root@SHAKTI:~/fabric-samples#
```

Set Fabric environment variables

export PATH="$PATH:$HOME/fabric-samples/bin"

export FABRIC_CFG_PATH="$HOME/fabric-samples/config"

```
root@SHAKTI:~/fabric-samples# export PATH="$PATH:$HOME/fabric-samples/bin"
root@SHAKTI:~/fabric-samples# export FABRIC_CFG_PATH="$HOME/fabric-samples/config"
root@SHAKTI:~/fabric-samples#
```

Verify Fabric version:

peer version

```
root@SHAKTI:~/fabric-samples# peer version
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

root@SHAKTI:~/fabric-samples#
```

Start the Fabric network and create a channel:

```
cd ~/fabric-samples/test-network
ls -l network.sh
./network.sh up createChannel
```

```
root@SHAKTI:~/fabric-samples# cd ~/fabric-samples/test-network
root@SHAKTI:~/fabric-samples/test-network# ls -l network.sh
-rwxr-xr-x 1 root root 23299 Oct 25 10:25 network.sh
root@SHAKTI:~/fabric-samples/test-network# ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds an
d using database 'leveldb with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
/root/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizat
ions
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizat
ions
org2.example.com
```

Deploy the JavaScript chaincode:

```
./network.sh deployCC \
 -ccn basic \
 -ccp ../asset-transfer-basic/chaincode-javascript \
 -ccl javascript
```

```
root@SHAKTI:~/fabric-samples/test-network# ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
```

Configure the Org1 peer environment:

```
root@SHAKTI:~/fabric-samples/test-network# export FABRIC_CFG_PATH="$HOME/fabric-samples/config"
root@SHAKTI:~/fabric-samples/test-network# export PATH="$PATH:$HOME/fabric-samples/bin"
root@SHAKTI:~/fabric-samples/test-network# export CORE_PEER_TLS_ENABLED=true
root@SHAKTI:~/fabric-samples/test-network# export CORE_PEER_LOCALMSPID="Org1MSP"
root@SHAKTI:~/fabric-samples/test-network# export CORE_PEER_TLS_ROOTCERT_FILE="$HOME/fabric-samples/test-netwo
rk/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
root@SHAKTI:~/fabric-samples/test-network# export CORE_PEER_MSPCONFIGPATH="$HOME/fabric-samples/test-network/o
rganizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp"
root@SHAKTI:~/fabric-samples/test-network# export CORE_PEER_ADDRESS=localhost:7051
root@SHAKTI:~/fabric-samples/test-network#
```

Initialize the ledger:

```
root@SHAKTI:~/fabric-samples/test-network#
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tl
sca.example.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/c
a.crt" \
  -c '{"function":"InitLedger","Args":[]}'
2026-10-25 10:54:21.541 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. re
sult: status:200
root@SHAKTI:~/fabric-samples/test-network#
```

Query All Assets:

```
root@SHAKTI:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":40
0,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"
asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"
Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"do
cType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
root@SHAKTI:~/fabric-samples/test-network#
```

Read a specific asset:

```
root@SHAKTI:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
root@SHAKTI:~/fabric-samples/test-network#
```

Create a new asset:

```
root@SHAKTI:~/fabric-samples/test-network#
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exa
mple.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
\
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt"
\
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset12","blue","5","Shakti","300"]}'
2026-10-25 11:08:14.649 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [bd164ccabd0f82534a69fdc3a036cb15266205d7214b
93097ae55d5ebee5c939] committed with status (VALID) at localhost:9051
2026-10-25 11:08:14.649 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [bd164ccabd0f82534a69fdc3a036cb15266205d7214b
93097ae55d5ebee5c939] committed with status (VALID) at localhost:7051
2026-10-25 11:08:14.650 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: s
tatus:200 payload:"{\"ID\":\"asset12\",\"Color\":\"blue\",\"Size\":5,\"Owner\":\"Shakti\",\"AppraisedValue\":300}"
root@SHAKTI:~/fabric-samples/test-network#
```

Read back the newly created asset:

```
root@SHAKTI:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset12"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset12","Owner":"Shakti","Size":5}
root@SHAKTI:~/fabric-samples/test-network#
```

Transfer ownership of the asset:

```
root@SHAKTI:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exa
mple.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
\
  -c '{"function":"TransferAsset","Args":["asset12","Riya"]}'
2026-10-25 11:10:30.018 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: s
tatus:200 payload:"Shakti"
root@SHAKTI:~/fabric-samples/test-network#
```

Shut down the network:

```
root@SHAKTI:~/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container orderer.example.com         Removed                                                    0.8s
 ✔Container peer0.org1.example.com      Removed                                                    1.6s
 ✔Container peer0.org2.example.com      Removed                                                    1.5s
 ✔Volume compose_peer0.org2.example.com Removed                                                    0.1s
 ✔Volume compose_orderer.example.com    Removed                                                    0.1s
 ✔Volume compose_peer0.org1.example.com Removed                                                    0.1s
 ✔Network fabric_test                   Removed                                                    0.4s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc149761
5f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:5c7966eaf9e4f9f46618728361b88200bbe68dc5d4dab22a76d97a7755a2b7a1
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac
0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:4c1060a07e654e8118624d503045ab3fc3bba476bb900982be4e8ca94d05cce8
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
37bb94b0940b: Pull complete
46500c34ad0c: Download complete
Digest: sha256:fd7dc98638c8e305f4dc34e979f1c0fdfdcaeb0fbf8fcff77ae834b6da3d7e6e
Status: Downloaded newer image for busybox:latest
root@SHAKTI:~/fabric-samples/test-network#
```

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.